



13file-systems

İşletim sistemi, o karmaşık disk bloklarını alıp bizim için nasıl "Klasörlere" ve "Dosyalara" dönüştürüyor?

Kullanıcı Gözünden Dosya Sistemi

İşletim sistemlerinin temel amacı, donanımın çirkinliklerini bizden saklamaktır. Sen bir Word belgesi açtığında diskin hangi sektörünün döndüğünü veya hangi bloğa yazıldığını umursamazsın; sen sadece "Dosya" adını verdiğimiz soyut bir kavramla ilgilenirsin.

Dosya (File) Aslında Nedir?

- Bilgisayarın gözünde dosya, ardışık mantıksal adreslerden oluşan bir bütündür.
- İşletim sistemi, kullanıcının hayatını kolaylaştırmak için her dosyaya bir kimlik kartı (Attributes) tanımlar. Dosyanın adı (insanların okuyabildiği tek şeydir), boyutu, diskteki konumu ve değiştirilme tarihi gibi bilgiler bu kimlik kartında tutulur.
- Bu kimlik bilgileri dosyanın kendi içinde değil, diskte tutulan "Dizin (Directory) Yapısında" saklanır.

Dosyaya Erişim Yöntemleri

Sıralı Erişim (Sequential Access)

Tıpkı eski bir kaset çalar gibidir. İşlemci, veriyi baştan sona doğru sırayla okur ("Sonrakini Oku", "Sonrakini Yaz", "Başa Sar" komutları vardır). En yaygın olanıdır.

Doğrudan / Rastgele Erişim (Direct Access)

Bir veritabanı düşün; 5 milyonuncu satırdaki veriyi bulmak için baştaki 4.999.999 satırı okumak tam bir eziyettir. Doğrudan erişimde, işletim sistemine "Bana direkt N. bloğu getir" dersin ve nokta atışı o veriye gidersin.

Dizinler (Directories / Klasörler)

Diyelim ki diskinde 10.000 tane dosya var. Kullanıcıların bu dosyaların diskteki fiziksel adreslerini ezberlemesi imkansızdır. Bize insan dilinde, anlaşılır isimler lazımdır. Bunun için klasörleme sistemleri tarih boyunca şöyle evrimleşmiştir:

- **Tek Seviyeli (Single-Level):** İlk başlarda tüm dosyalar aynı devasa odanın (klasörün) içindeydi. İki kişi aynı ismi kullanamazdı, tam bir isimlendirme kabusuydu.
- **İki Seviyeli (Two-Level):** Her kullanıcıya kendi özel odası verildi. "Mertcan'ın odasındaki test.txt" ile "Ahmet'in odasındaki test.txt" artık karışmıyordu. Ama dosyaları gruplamak hala imkansızdı.
- **Ağaç Yapısı (Tree-Structured):** Bugün Windows ve Linux'ta kullandığımız o harika yapıdır. Kök (Root) bir klasörden başlar ve klasörlerin içine sınırsız sayıda alt klasör açmana izin verir.
- **Acyclic ve Genel Graflar:** Ağaç yapısı harikadır ama ya bir dosyaya iki farklı klasörden *aynı anda* erişmek istersen? İşte o zaman Kısayollar/Bağlantılar (Links/aliasing) icat edilir. Böylece aynı dosya, birden fazla klasörün içinde yaşıyormuş gibi görünebilir.

Bağlantılar (Links) - Aynı Dosyaya Birden Fazla İsim Vermek

Sert Bağlantı (Hard Link)

- Aynı inode'a işaret eden ikinci bir dizin girişi oluşturulur.
- Unix, inode içinde bir **bağlantı sayacı (link count)** tutar. Her hard link eklediğinizde sayaç 1 artar, silindiğinde azalır. Sayaç 0 olunca dosya gerçekten silinir.
- **Sorun:** Döngüsel bağlantılar? Dizinlere hard link yapılmaz (genelde). Sadece dosyalara izin verilir.

Sembolik Bağlama (Soft Link - Symlink)

- Ayrı bir dosya gibidir; içinde hedef dosyanın **adını** tutar (inode numarası değil).
- Hedef silinirse, sembolik bağlantı "kırık" olur (dangling pointer).
- Dizinlere de sembolik bağlantı yapılabilir.
- Dosya sistemi bir sembolik bağlantıya rastlayınca otomatik olarak hedefe yönlendirir.

Özet:

- Hard link → aynı dosyaya farklı isimler (sayaçlı).
- Soft link → bir kısayol dosyası (hedef adı yazılı).

Güvenlik ve Koruma

Aynı bilgisayarı birden fazla kişi kullanıyorsa dosyalar korunmalıdır. Unix (Linux) sistemleri bunu 3 yetki ve 3 kullanıcı sınıfıyla çözer:

- **Yetkiler:** Okuma (Read - R), Yazma (Write - W), Çalıştırma (Execute - X).
- **Sınıflar:** Sahibinin (Owner), Grubun (Group) ve Geri Kalan Herkesin (Public) yetkileri ayrı ayrı belirlenir.
 - Sınavlarda sıkça çıkan o meşhur `chmod 761` komutunun sırrı buradadır. Bu komut sırasıyla Sahip, Grup ve Herkes için 3 bitlik (ikilik tabanda) izinleri belirler.

Klasör ve Dosyaların Okunup Yazılması

Sen C'de veya Python'da `open("oyun.exe")` yazdığında, bu emir doğrudan diske gitmez. İşletim sistemi bu emri 4 farklı çevirmenden geçirir:

- **Mantıksal Dosya Sistemi (Logical File System):** "oyun.exe" ismini alır ve bunun diskteki gizli kimlik numarasına (Inode numarasına) çevirir. Klasör yetkilerini ve şifreleri kontrol eden polis burasıdır.
- **Dosya Organizasyon Modülü:** "Dosyanın 5. bloğunu istiyorum" emrini alır ve bunu fiziksel adrese ("Disk 2, Silindir 72, Sektör 10") dönüştürür.

- **Temel Dosya Sistemi (Basic File System):** Aygıt sürücüsüne "Bana şu adresteki bloğu getir" der. Aynı zamanda RAM'deki önbellekleri (Cache/Buffer) yönetir.
- **I/O Control:** İşin en altındaki ameledir (Device Drivers). Komutları 0 ve 1'lere, elektriksel sinyallere çevirip diskin motoruna yollar.

FCB (Dosya Kontrol Bloğu) ve Açık Dosyalar Masası

Bir dosyaya çift tıkladığında, işletim sistemi her okuma/yazma işlemi için diske gitmez (çünkü disk RAM'e göre milyonlarca kat yavaştır).

- Diskteki **FCB'yi (File Control Block / Inode)** bulur. FCB; dosyanın boyutu, sahibinin kim olduğu ve bloklarının nerede olduğunu tutan bir nevi kimlik kartıdır.
- İşletim sistemi bu FCB'yi kopyalar ve RAM'in (Çekirdek Belleğinin) içinde **"Sistem Geneli Açık Dosyalar Masası"na (System-wide open-file table)** koyar.
- Böylece sen dosyayı okurken doğrudan RAM'deki bu masaya bakarsın. Dosyayı kapattığında (Close) RAM'deki kopyası silinir ve değişiklikler diske geri yazılır.

Klasörlerin Diske Yazılması

İşin ilginç yanı şudur: İşletim sistemi için **Klasör (Directory) diye bir şey yoktur; klasörler de aslında özel birer dosyadır!** Klasör dosyasının içinde veriler değil, sadece "İsim ve Kimlik Numarası (Inode)" listeleri bulunur. Bu listeyi iki şekilde tutar:

- **Düz Liste (Linear List):** İsimler alt alta yazılır. Programlaması çok basittir ama klasörde 10.000 dosya varsa, aradığını bulmak için baştan sona taramak (Linear Search) bilgisayarı felç eder.
- **Hash Tablosu:** Aranılan dosyayı anında bulmamızı sağlayan matematiksel bir arama yöntemidir. Çok hızlıdır ama Hash çakışmaları (Collisions) gibi yazılımsal dertleri vardır.

Dosyaların Diske Yerleşmesi

1. Bitişik Tahsis Yöntemi (Contiguous Allocation)

En ilkel ve en hızlı yöntemdir. Dosyanın tüm blokları diskte sırayla, yan yana dizilir.

- **Avantajı:** İnanılmaz hızlıdır (Sıralı erişim). Disk kolu (iğne) bir kere okumaya başladı mı hiç sağa sola savrulmadan tüm dosyayı tek nefeste okur.
- **Dezavantajı (Dış Parçalanma):** Diskte 100 tane boş kutu olsa bile, bunlar yan yana değilse (aralara başka dosyalar girmişse) o 5 kutuluk dosyayı kaydedemezsin! Ayrıca dosyayı sonradan büyütme istersen ve yanındaki kutu doluyorsa, bütün dosyayı alıp başka geniş bir yere taşımak zorunda kalırsın.

(Günümüzde APFS, NTFS ve ext4 gibi modern sistemler "Extent" adı verilen değiştirilmiş, esnek bir bitişik tahsis kullanır.)

2. Bağlı Tahsis (Linked Allocation)

Bitişik tahsisin "yan yana" kuralından bıkan mühendisler, dosyayı parçalayıp diskin her yerine rastgele saçarlar.

Tıpkı bir hazine avı gibi; 1. kutunun içine veriyi koyarlar, bir de 2. kutunun nerede olduğunu gösteren bir "Ok/İşaretçi" (Pointer) koyarlar.

- **Avantajı:** Diskte neresi boşsa orayı kullanırsın, dış parçalanma (yer israfı) sıfırdır. Dosya istediği kadar büyüyebilir.
- **Dezavantajı:** Baştan sona okumak harikadır ama **Rastgele Erişim (Direct Access) bir kabustur!** Sen "Bana dosyanın 100. bloğunu getir" dersin, işletim sistemi mecburen 1'den başlayıp okları takip ederek 99 tane bloğu boş yere okumak zorunda kalır. Çok yavaştır.

FAT (Dosya Tahsis Tablosu - File Allocation Table)

Hazine avındaki o "okları" kutuların içinden çıkarıp, diskin en başına koyduğumuz tek bir "Fihrist/Tablo" içinde toplamaktır.

Eski DOS ve Windows sistemleri (USB belleklerdeki FAT32) bunu kullanır.

- Bütün oklar bu tabloda olduğu için, işletim sistemi bu tabloyu hızlıca RAM'e (Önbelleğe) alır. Böylece 100. bloğu bulmak için diski değil, RAM'deki tabloyu tarar ve nokta atışı diske gider. Çok daha akıllıcadır.

Dizinli/İndeksli Tahsis (Indexed Allocation) ve Unix Inode

Günümüzdeki modern Linux/Unix sistemlerinin (UFS, ext) kullandığı asıl kusursuz yapıdır. Her dosya için "İçindekiler" (Index) sayfası niteliğinde özel bir blok yaratılır. Buna **Inode** denir.

Dosyanın tüm verileri diske rastgele saçılır ama bu verilerin nerede olduğu tek tek bu Inode bloğunun içine yazılır.

Peki ya dosya çok büyükse ve içindekiler sayfası tek bir Inode bloğuna sığmazsa ne olur? İşte burada **Çok Seviyeli İndeks (Multi-Level Index)** devreye girer:

- **Doğrudan (Direct) İşaretçiler:** İlk 12 blok için adresler doğrudan Inode'un içindedir.
- **Tekli Dolaylı (Single Indirect):** Dosya büyürse, Inode yeni bir bloğu işaret eder. Bu yeni bloğun içi veri değil, *diğer veri bloklarının adresleriyle* doludur.
- **Çift (Double) ve Üçlü (Triple) Dolaylı:** Dosya gigabaytlar veya terabaytlar boyutundaysa, işaretçiler "Adresleri tutan blokların adreslerini tutan bloklara" (Inception gibi) işaret eder. Bu sayede tek bir Inode ile devasa dosyalar kayıpsız yönetilir.

FAT ve Inode Farkı

- **FAT:** Bellekteki bütün dosyaları bir linked listte tutar, bir dosyanın 10 bininci bloğuna ulaşmak için önceki blokların node'larını teker teker gezmeniz gerekir.
- **Inode:** Her dosya için ayrı bir dizi oluşturur, bir dosyanın herhangi bir bloğunun bulunduğu elemana direkt ulaşabilirsiniz.

Performans Sorunları ve BSD Fast File System (FFS)

1970'lerin sonundaki orijinal Unix dosya sistemi **çok yavaştı**. Öyle ki, diskin teorik hızının sadece %2'sini kullanabiliyordu (20 KB/sn gibi). Şimdi bu yavaşlığın nedenlerini ve BSD'nin nasıl **FFS (Fast File System)** ile bu sorunu çözdüğünü anlayacağız.

Neden orijinal Unix FS bu kadar yavaştı?

1. Bloklar çok küçüktü.

- Küçük blok / dosyalar için daha az iç parçalanma (internal fragmentation) ama **her blok erişiminde aynı sabit süre** (seek + rotation) ödeniyor.
- Büyük dosyaları okurken milyonlarca blok başına aynı gecikme tekrarlanıyor → çok yavaş aktarım hızı.

2. İlişkili veriler birbirinden uzaktı.

- Bir dosyanın blokları ardışık değil, diskin dört bir yanına dağılmış.
- Dosyanın inode'u ile veri blokları farklı silindirlerde.
- Aynı dizindeki dosyaların inode'ları birbirinden uzakta.

3. Serbest blok listesi bağlı liste (linked list) şeklindeydi

- Yeni blok ayrılırken listenin başından alınıyordu.
- Zamanla bu liste rastgele sıralanıyor, ardışık blok bulmak imkansız hale geliyordu.

4. Tüm metadata güncellemeleri senkron (synchronous) yapılıyordu

- Her yazma işlemi, veri diske sağlam yazılana kadar bekliyordu. Bu çok yavaş.



İkilem: Büyük blok hızlı ama çok israf; küçük blok israfsız ama yavaş.

FFS Çözümü: Blok + Parça (Fragment) Sistemi

- **Büyük blok** (4 KB veya 8 KB) kullan → yüksek aktarım hızı.

- Bir dosyanın son kısmı veya çok küçük dosyalar için bu büyük bloğu **daha küçük parçalara (fragments)** böl (1 KB, 2 KB gibi).
- Böylece hem hızlı ardışık erişim hem de az iç parçalanma.

Örnek: 8 KB blok, 1 KB fragment. 10 KB'lık bir dosya: 8 KB'lık bir tam blok + 2 KB'lık bir fragment olarak yer kaplar.

Silindir Grupları (Cylinder Groups)

Bir HDD'de fiziksel olarak şunlar vardır:

- **İz (track):** Dönen plaka üzerinde aynı yarıçapa sahip daire şeklindeki alan.
- **Silindir (cylinder):** Aynı yarıçaptaki tüm plakaların üzerindeki izlerin düşeyde hizalanmasıyla oluşan küme. Yani birden fazla plaka varsa, her plakanın aynı konumundaki izlerin toplamına silindir denir.
- **Sektör (sector):** Bir izin bölündüğü en küçük adreslenebilir birim (genelde 512 byte veya 4 KB).

Silindir grubu (cylinder group) ise FFS'nin bir **sanal organizasyonudur**. Fiziksel olarak birbirine çok yakın birkaç silindiri alıp, onları bir grup olarak ele alır. Bu grup içindeki herhangi bir sektöre erişmek için disk kafasının **silindirler arası hareket etmesi gerekmez**; sadece aynı silindir içinde iz değiştirme (daha hızlı) veya rotasyon bekleme yeterlidir. Grup dışındaki bir silindire gitmek için ise kafa hareketi (seek) gerekir.

Nasıl Çalışır?

- Bir dosyanın inode'unu ve veri bloklarını **aynı silindir grubuna** koymaya çalış.
- Aynı dizindeki tüm dosyaların inode'larını (ve mümkünse verilerini) **aynı silindir grubuna** koy.
- Farklı dizinleri **farklı silindir gruplarına** yerleştir.

Sonuç:

- `ls -l` yapınca hemen hemen hiç seek yok → çok hızlı.
- Ardışık dosya okumaları neredeyse hiç kafa hareketi gerektirmiyor.

Serbest blok yönetimi: Bitmap'e geçiş

Bitmap, basitçe bir dizi bit'tir. Her bir bit, disk üzerindeki bir bloğun **dolu mu yoksa boş mu** olduğunu gösterir. Kural genellikle şöyledir:

- 1 = blok kullanılıyor (dolu),
- 0 = blok boş (yeni bir dosya veya dosya uzantısı için kullanılabilir).

Neden Bitmap Kullanılır?

Eski Unix'te serbest bloklar bir bağlı liste (linked list) ile tutuluyordu. Listenin başından bir blok alınıyor, kullanılıyor, silinince geri listeye ekleniyordu. Zamanla liste rastgele sıralanıyor, ardışık bloklar birbirini izlemiyordu. Bu da büyük dosyaların parçalanmasına (fragmentation) ve yavaş okuma/yazmaya yol açıyordu.

Bunun yerine artık bu kontrol, bitmap denilen ve diskteki toplam blok sayısı kadar bitten oluşan bir dizi üzerinden yapılır.

Çökme Sonrası Tutarlılık

Şimdi dosya sistemlerinin en can alıcı sorunlarından birine geliyoruz: **Elektrik giderse, sistem çökerse veya reset atarsak ne olur?**

Dosya sistemi her zaman tutarlı (consistent) kalmalı ama disk yazmaları anlık değildir. Bir dosya oluştururken birden fazla disk bloğu güncellenir. Eğer bu güncellemelerin yalnızca bir kısmı diske yazılıp çökme olursa, dosya sistemi **bozulabilir** (tutarsız hale gelir).

Bu bölümde, eski çözüm olan `fsck` 'ten başlayıp, daha gelişmiş yöntemler olan **soft updates** ve **journaling (günlükleme)** 'e kadar adım adım gideceğiz.

1. Eski yaklaşım: fsck (File System Check)

Çökme olduktan sonra, yeniden başlatma sırasında tüm diski tarayarak hataları bul ve düzelt. Bu işi yapan programa `fsck` denir.

fsck nasıl çalışır?

1. **Superblock kontrolü:** Superblock (disk hacminin başlangıcındaki bilgi bloğu) makul görünüyor mu? Örneğin, dosya sisteminin boyutu, ayrılmış blok sayısından büyük mü gibi temel tutarlılıklar kontrol edilir.
2. **Serbest blokların yeniden oluşturulması:** fsck, tüm inode'ları, dolaylı blokları (indirect blocks) ve çift dolaylı blokları tarayarak **hangi blokların gerçekten kullanıldığını** belirler. Sonra bu bilgiyi kullanarak, bitmap'lerdeki hataları düzeltir. Eğer bitmap ile inode'lar arasında tutarsızlık varsa, **inode'lara güvenilir** ve bitmap onlara göre yenilenir.
3. **Inode durumu:** Her inode'un tutarlılığı kontrol edilir. Örneğin, bir inode'un blok numaraları disk sınırları içinde mi? Aynı blok iki farklı dosyaya mı ait (çapraz tahsis)? Eğer bir inode anlamsız veriler içeriyorsa, temizlenir.
4. **Bağlantı sayacı (link count) düzeltmesi:** Bir dosyaya kaç tane dizin girişi (hard link) olduğu sayılır. Bu sayı, inode'daki link count ile eşleşmezse, inode'daki sayı düzeltilir. Eğer link count sıfırdan büyük ama hiçbir dizin girişi yoksa, dosya **lost+found** klasörüne taşınır.
5. **Dizin kontrolü:** Dizin dosyalarında **.** ve **..** gibi özel girişlerin geçerli olup olmadığı, dosya isimlerinin benzersiz olup olmadığı, dizinde "delik" (geçersiz boşluk) bulunup bulunmadığı kontrol edilir. Tüm dizinlerin kök dizinden itibaren erişilebilir olduğu teyit edilir.

fsck'nin sorunları

- Çok **yavaştır**. Büyük disklerde fsck dakikalar, hatta saatler sürebilir.
- fsck ancak **bozulmayı tespit edip düzeltebilir**; veri kaybını önleyemez.

Bu yüzden daha iyi yöntemler geliştirilmiştir: **Ordered updates**, **Soft updates** ve **Journaling**.

Ordered Updates (Sıralı Güncellemeler)

Kural 1: Asla bir pointer'ı, gösterdiği yapı başlatılmadan diske yazma.

Örnek: Bir dosyanın inode'unu yazmadan önce, o inode'un içindeki blok pointer'larının geçerli olması gerekir. Ya da bir dizin girişi, inode numarasını göstereceği için, önce inode diske yazılmalı, sonra dizin girişi yazılmalı.

Kural 2: Bir kaynağı, ona olan tüm pointer'lar silinmeden tekrar kullanma.

Örnek: Bir dosya silindiğinde, önce dizin girişi kaldırılmalı, sonra inode'daki link count azaltılmalı veya inode serbest bırakılmalı. Eğer sıra tersine çökerse, silinmiş dosyaya hâlâ işaret eden bir dizin girişi kalabilir ve bu dosya yeniden kullanıldığında çakışma olur.

Kural 3: Bir kaynağa giden son pointer'ı, yeni bir pointer oluşturmadan temizleme.

Örnek: Bir dosyanın taşınması veya yeniden adlandırılması gibi durumlarda, önce yeni bağlantı oluşturulur, sonra eski bağlantı silinir. Böylece çökme anında her zaman en az bir geçerli yol bulunur.

Soft Updates

Temel fikir: **Blokları istediğiniz sırada yazabilirsiniz, ancak yazarken bağımlılıkları ihlal eden değişiklikleri geçici olarak geri alarak (rollback) bloğu güvenli hale getirirsiniz.**

Diyelim ki aynı dizin bloğu (Y) ve inode bloğu (X) üzerinde iki işlem yapıyoruz:

- A dosyasını oluştur (inode'u X'te bir giriş, dizinde Y'de bir isim)
- B dosyasını sil (inode'u X'te işaretle, dizinde Y'den ismi kaldır)

Normalde, oluşturma Y→X bağımlılığı, silme ise X→Y bağımlılığı gerektirir (çünkü önce dizin girişini silmelisin). Bu bir döngüdür.

Soft updates'te şöyle yapılır:

- Bellekte her iki değişiklik de yapılır (A eklendi, B silindi).
- Disk yazma zamanı geldiğinde, örneğin Y bloğunu yazmak istiyoruz. Ama Y'nin içinde A'nın girişi var ve bu giriş, henüz diske yazılmamış olan X inode bloğuna bağımlıdır (Kural 1). Bu yüzden Y bloğuna yazmadan önce, A'ya ait olan girişi **geçici olarak geri alırsız** (sanki A hiç oluşturulmamış gibi yaparsız). Sonra Y bloğunu diske yazarız. Artık Y diske yazıldı, içinde sadece B'nin silinmiş hali var (B zaten yok, sorun yok).

- Şimdi **X** bloğuna bakalım. **X** 'te A'nın inode'u var, B'nin inode'u işaretlenmiş. **X** 'in hiçbir bağımlılığı kalmadı (çünkü **Y** 'de A girişi yok). **X** 'i rahatça diske yazarız.
- **X** diske yazıldıktan sonra, **Y** bloğu bellekte hâlâ kirli (dirty) çünkü A'nın girişini daha önce geri almıştık. Şimdi A'nın girişini tekrar **Y** 'ye ekleriz (çünkü bağımlılık ortadan kalktı, **X** diskte) ve **Y** 'yi ikinci kez diske yazarız.

Journaling (Günlükleme)

Günümüzde en yaygın kullanılan yöntem journaling'dir (ext3, ext4, NTFS, JFS, XFS vb.).

Temel fikir: Metadata işlemlerini (dosya oluşturma, silme, yeniden adlandırma gibi) önce diskin ayrılmış küçük bir bölgesine (log veya journal) sırayla yaz. İşlem log'a yazıldıktan sonra "tamamlandı" say. Daha sonra, arka planda bu işlemleri asıl disk konumlarına uygula. Çökme olursa, log'taki işlemleri yeniden oynatarak (replay) eksik kalanları tamamla.

Journaling'in Avantajları

- fsck'ye gerek kalmaz veya çok kısa sürer (sadece log'u kontrol eder).
- Performansı iyidir çünkü log'a yazma ardışık olduğu için çok hızlıdır (seek yok). Asıl yazmalar arka planda optimize edilebilir.
- Veri kaybını önlemez (sadece metadata tutarlılığını sağlar; kullanıcı verisinin kaybolması mümkündür), ancak dosya sistemi bozulmaz.

Journaling Türleri

- **Metadata journaling:** Sadece inode, dizin, bitmap gibi metadata'lar loglanır. Dosya içerikleri loglanmaz. Daha hızlıdır, yaygın olarak kullanılır (ext3, ext4 varsayılan).
- **Data journaling:** Hem metadata hem de dosya verileri loglanır. Daha güvenli ama daha yavaştır (çünkü veri de log'a yazılır, iki kez yazma olur).

Flash-Friendly File System (F2FS)

F2FS, özellikle **flash bellekler (SSD'ler, USB bellekler, SD kartlar)** için tasarlanmış bir dosya sistemidir.

Flash belleklerin geleneksel sabit disklerden (HDD) farklı özellikleri vardır: rastgele erişim çok hızlıdır, ancak **üzerine yazma** işlemleri sınırlıdır (aşınma ömrü). F2FS, bu özellikleri göz önüne alarak performansı ve ömrü artırmayı hedefler.



Not: F2FS, flash cihazların içindeki "FTL (Flash Translation Layer)" ile birlikte çalışır. FTL, flashın fiziksel sınırlarını gizler ama F2FS yine de flasha uygun stratejiler kullanır.

F2FS'in Temel Fikri

1. Bütük ve ardışık yazmalar yapmaya çalış

Flash belleklerde küçük rastgele yazmalar, büyük ardışık yazmalardan daha yavaştır ve daha fazla aşınmaya yol açar. F2FS, mümkün olduğunca verileri **büyük parçalar halinde ve ardışık olarak** yazar.

2. Aşınma dengeleme (wear leveling) yapmaz

Flash cihazların içindeki FTL zaten aşınma dengeleme yapar. F2FS bunu tekrar yapmaz, sadece FTL'nin işini kolaylaştıracak şekilde yazma düzeni oluşturur.

3. Diski hiyerarşik birimlere böler

F2FS, disk şu şekilde kademeli birimlere ayırır:

- **Blok (Block):** En küçük birim, 4 KB (tıpkı sayfa boyutu gibi).
- **Segment (Segment):** 512 bloktan oluşur (toplam 2 MB). Segment, F2FS'nin temel yazma birimidir. Segmentin içindeki her blok hakkında özet bilgi (segment summary) tutulur.
- **Bölüm (Section):** 2^i tane segmentten oluşur (varsayılan olarak $i=0$, yani 1 segment). Bölüm, log temizleme (log cleaning) işleminin birimidir.
- **Bölge (Zone):** n tane bölümden oluşur (varsayılan $n=1$). Cihazın içinde birden fazla alt cihaz (subdevice) varsa, farklı bölgelere paralel G/Ç yapılabilir.

4. Cihazı iki ana bölgeye ayırır

- **Ana alan (Main area):** Büyük ardışık yazmaların yapıldığı yer. Verilerin ve metadata'nın büyük kısmı buraya yazılır.
- **Küçük metadata alanı:** Rastgele yazmaların olduğu yer (örn. bit haritaları, segment özetleri). Bu alan daha küçüktür ve FTL'nin rastgele yazma performansına güvenir.

F2FS'nin Özel Yapıları

- **CP (Checkpoint):** Sistemin tutarlı bir anlık görüntüsünü tutar. İki adet CP alanı vardır, biri aktifken diğeri pasiftir. Her CP'de bir sürüm numarası bulunur. Çökme sonrası en yüksek sürümlü tutarlı CP kullanılır. Ayrıca, açıkken silinmiş dosyaların (orphan inodes) listesini de tutar.
- **SIT (Segment Information Table):** Her segment için bir bit haritası (bitmap) tutar; segment içinde hangi blokların geçerli (dolu) olduğunu gösterir. Ayrıca her segmentteki geçerli blok sayısını (valid block count) saklar. İki SIT alanı ve küçük bir günlük (journal) ile yerinde güncellemeden (in-place update) kaçınılır.
- **NAT (Node Address Table):** Düğüm (node) numaralarını (inode benzeri) bu düğümün diskteki gerçek blok adresine çevirir. SIT gibi güncellenir. Bu sayede bir düğüm taşındığında, tüm üst pointer'ların güncellenmesi gerekmez; sadece NAT güncellenir (bir seviye dolaylılık).
- **SSA (Segment Summary Area):** Her blok için ebeveyn bilgisi tutar (hangi inode'un hangi offset'i). Sadece yerinde güncellenir. CP, hangi SSA'ların aktif olduğunu kaydeder.